TP 2: Modelado y Relaciones BDD No Relacional

Alumno: Sussini, Patricio

Materia: Bases de Datos 2

Actividad 1:

1) Plataforma educativa:

a) Estructura JSON y Diagrama:

i) Colección **Cursos** contiene datos básicos y un resumen embebido de las evaluaciones para acceso rápido.

```
{
  "_id": ObjectId("65f1a..."),
  "nombre": "Programación II",
  "docente": "Alberto Sussini",
  "resumen_evaluaciones": {
    "promedio_general": 8.5,
    "cantidad_total": 45
  }
}
```

ii) Colección **Evaluaciones** contiene el detalle completo y referencia al curso.

```
{
  "_id": ObjectId("..."),
  "curso_id": ObjectId("65f1a..."),
  "estudiante": "Juan Pérez",
  "nota": 9,
  "fecha": "2026-03-20"
}
```

b) Justificación del Patrón Híbrido:

i) **Rendimiento:** Permite mostrar estadísticas rápidas de promedio y cantidad al listar cursos sin tener que consultar miles de evaluaciones detalladas.

ii) **Escalabilidad:** Evita que el documento del curso crezca indefinidamente si hay demasiados estudiantes, ya que los detalles están en otra colección.

2) Creación en MongoSH:

a) Paso a paso de ejecución de comandos.

```
Atlas atlas-u9mhvu-shard-0 [primary] test> use plataforma_educativa
switched to db plataforma_educativa
Atlas atlas-u9mhvu-shard-0 [primary] plataforma_educativa> S
```

Crear colección cursos con datos embebidos

```
Atlas atlas-u9mhvu-shard-0 [primary] plataforma_educativa> db.cursos.insertMany([
|   {
|     nombre: "Base de Datos NoSQL",
|     cuatrimestre: 1,
|     resumen_evaluaciones: { promedio: 7.5, cantidad: 2 }
|   },
|   {
|     nombre: "Programación IoT",
|     cuatrimestre: 2,
|     resumen_evaluaciones: { promedio: 9.0, cantidad: 1 }
|   }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69cf3d87741aa807bf3682d1'),
    '1': ObjectId('69cf3d87741aa807bf3682d2')
  }
}
```

Obtener los IDs generados para referenciarlos abajo

```
Atlas atlas-u9mhvu-shard-0 [primary] plataforma_educativa> const nosqlId = db.cursos.findOne({nombre: "Base de Datos NoS
QL"})._id;
| const iotId = db.cursos.findOne({nombre: "Programación IoT"})._id;
```

Crear colección evaluaciones con referencias

```
Atlas atlas-u9mhvu-shard-0 [primary] plataforma_educativa> db.evaluaciones.insertMany([
|   { curso_id: nosqlId, estudiante: "Carlos", nota: 8, fecha: "2026-03-01" },
|   { curso_id: nosqlId, estudiante: "Ana", nota: 7, fecha: "2026-03-02" },
|   { curso_id: iotId, estudiante: "Beatriz", nota: 9, fecha: "2026-03-05" }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69cf3dd1741aa807bf3682d3'),
    '1': ObjectId('69cf3dd1741aa807bf3682d4'),
    '2': ObjectId('69cf3dd1741aa807bf3682d5')
  }
}
```

Consulta de curso específico con resumen

```
Atlas atlas-u9mhvu-shard-0 [primary] plataforma_educativa> db.cursos.find({ nombre: "Base de Datos NoSQL" })
[
  {
    _id: ObjectId('69cf3d87741aa807bf3682d1'),
    nombre: 'Base de Datos NoSQL',
    cuatrimestre: 1,
    resumen_evaluaciones: { promedio: 7.5, cantidad: 2 }
  }
]
```

Consultar todas las evaluaciones de un curso específico

```
Atlas atlas-u9mhvu-shard-0 [primary] plataforma_educativa> db.evaluaciones.find({ curso_id: nosqlId })
[
  {
    _id: ObjectId('69cf3dd1741aa807bf3682d3'),
    curso_id: ObjectId('69cf3d87741aa807bf3682d1'),
    estudiante: 'Carlos',
    nota: 8,
    fecha: '2026-03-01'
  },
  {
    _id: ObjectId('69cf3dd1741aa807bf3682d4'),
    curso_id: ObjectId('69cf3d87741aa807bf3682d1'),
    estudiante: 'Ana',
    nota: 7,
    fecha: '2026-03-02'
  }
]
```

Agregación para promedio de notas por curso

```
Atlas atlas-u9mhvu-shard-0 [primary] plataforma_educativa> db.evaluaciones.aggregate([
|   { $group: { _id: "$curso_id", promedioReal: { $avg: "$nota" } } }
| ])
[
  { _id: ObjectId('69cf3d87741aa807bf3682d1'), promedioReal: 7.5 },
  { _id: ObjectId('69cf3d87741aa807bf3682d2'), promedioReal: 9 }
]
```

Código en forma COPY PASTE:

```
// a) Crear estudiantes
db.estudiantes.insertMany([
  { nombre: "Lucas", carrera: "Programación" },
  { nombre: "Sofía", carrera: "Programación" },
  { nombre: "Mateo", carrera: "Sistemas" }
])

// b) Crear tutores
db.tutores.insertMany([
  { nombre: "Ing. García", especialidad: "Backend" },
  { nombre: "Lic. Rodríguez", especialidad: "Soft Skills" }
])

// Obtener IDs
const estId = db.estudiantes.findOne({nombre: "Lucas"})._id;
const tutId = db.tutores.findOne({nombre: "Ing. García"})._id;

// c) Crear colección asociativa 'tutorias'
db.tutorias.insertOne({
  estudiante_id: estId,
  tutor_id: tutId,
  fecha_inicio: "2026-03-15",
  tipo_acompanamiento: "técnico"
})

// d) Consultar todos los estudiantes de un tutor específico
// Primero buscamos los IDs en la tabla asociativa y luego podrías usar $lookup
db.tutorias.find({ tutor_id: tutId })
```

Actividad 2:

1) Estudiante-Tutor - Muchos a muchos

a) **Tipo de relación:** Muchos a muchos. Un estudiante tiene varios tutores y un tutor varios alumnos.

b) Patrón Recomendado: Colección Asociativa. Es la mejor opción cuando la relación tiene atributos propios que no pertenecen naturalmente ni al estudiante ni al tutor de forma aislada.

c) Diseño de documentos:

```
// Documento en colección 'tutorias'
{
  "estudiante_id": ObjectId("..."),
  "tutor_id": ObjectId("..."),
  "fecha_inicio": "2026-03-10",
  "tipo_acompanamiento": "técnico"
}
```

2) Implementación:

Crear estudiantes

```
Atlas atlas-u9mhvu-shard-0 [primary] tutorias> db.estudiantes.insertMany([
|   { nombre: "Lucas", carrera: "Programación" },
|   { nombre: "Sofía", carrera: "Programación" },
|   { nombre: "Mateo", carrera: "Sistemas" }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69cf4016741aa807bf3682d6'),
    '1': ObjectId('69cf4016741aa807bf3682d7'),
    '2': ObjectId('69cf4016741aa807bf3682d8')
  }
}
```

Crear tutores

```
Atlas atlas-u9mhvu-shard-0 [primary] tutorias> db.tutores.insertMany([
|   { nombre: "Ing. García", especialidad: "Backend" },
|   { nombre: "Lic. Rodríguez", especialidad: "Soft Skills" }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69cf403f741aa807bf3682d9'),
    '1': ObjectId('69cf403f741aa807bf3682da')
  }
}
```

Obtener IDs

```
Atlas atlas-u9mhvu-shard-0 [primary] tutorias> const estId = db.estudiantes.findOne({nombre: "Lucas"})._id; const tutId
= db.tutores.findOne({nombre: "Ing. García"})._id;
```

Crear colección asociativa 'tutorias'

```
Atlas atlas-u9mhvu-shard-0 [primary] tutorias> db.tutorias.insertOne({
|   estudiante_id: estId,
|   tutor_id: tutId,
|   fecha_inicio: "2026-03-15",
|   tipo_acompanamiento: "técnico"
| })
{
  acknowledged: true,
  insertedId: ObjectId('69cf409e741aa807bf3682db')
}
```

Consultar todos los estudiantes de un tutor específico

```
Atlas atlas-u9mhvu-shard-0 [primary] tutorias> db.tutorias.find({ tutor_id: tutId })
[
  {
    _id: ObjectId('69cf409e741aa807bf3682db'),
    estudiante_id: ObjectId('69cf4016741aa807bf3682d6'),
    tutor_id: ObjectId('69cf403f741aa807bf3682d9'),
    fecha_inicio: '2026-03-15',
    tipo_acompanamiento: 'técnico'
  }
]
```

Para filtrar y acomodar los resultados

```
[
  {
    fecha_inicio: '2026-03-15',
    tipo_acompanamiento: 'técnico',
    estudiante: 'Lucas',
    carrera: 'Programación'
  }
]
```

Código en forma COPY PASTE:

```
// a) Crear recursos compartidos
db.recursos.insertMany([
  { _id: 101, titulo: "Guia SQL", tipo: "PDF" },
  { _id: 102, titulo: "Intro a la Nube", tipo: "Video" }
])

// b) Crear materias que referencian recursos
db.materias.insertMany([
  { nombre: "Base de Datos I", recursos_ids: [101] },
  { nombre: "Sistemas Operativos", recursos_ids: [101, 102] }
])

// c) ¿En cuántas materias se usa el recurso 101?
db.materias.countDocuments({ recursos_ids: 101 })

// d) Listar materias que usan el recurso 101
db.materias.find({ recursos_ids: 101 }, { nombre: 1 })

// e) Agregar un nuevo recurso a una materia existente
db.materias.updateOne(
  { nombre: "Base de Datos I" },
  { $push: { recursos_ids: 102 } }
)
```

Actividad 3:

1) Biblioteca digital - Recursos compartidos

a) Patrón a usar: Referencia

b) No usamos embebido porque el recurso se utiliza en distintas materias. Si embebemos el PDF completo en cada materia, al actualizar el link del PDF tendríamos que buscar y editar múltiples documentos, generando inconsistencias y redundancia innecesaria.

c) Colección **recursos**, estructura de datos.

```
{
  "_id": 101,
  "titulo": "Guía de SQL Avanzado",
  "tipo": "PDF",
  "url": "https://biblioteca.utn.edu/docs/sql-adv.pdf",
  "tamano": "2.4MB"
}
```

Colección **materias**

```
{
  "_id": ObjectId("..."),
  "nombre": "Bases de Datos II",
  "codigo": "BD2026",
  "recursos_ids": [101, 105, 110]
}
```

2) Implementación Práctica

Crear la colección 'recursos' con datos reutilizables

```
Atlas atlas-u9mhvu-shard-0 [primary] plataforma_educativa> db.recursos.insertMany([
|   { _id: 101, titulo: "Manual de MongoDB", tipo: "PDF", link: "utn.edu/mongo.pdf" },
|   { _id: 102, titulo: "Video: Modelado NoSQL", tipo: "Video", link: "youtube.com/nosql" },
|   { _id: 103, titulo: "Whitepaper Cloud Computing", tipo: "PDF", link: "aws.com/cloud.pdf" }
| ])
{ acknowledged: true, insertedIds: { '0': 101, '1': 102, '2': 103 } }
```

Crear la colección 'materias' referenciando los IDs de los recursos

```
Atlas atlas-u9mhvu-shard-0 [primary] plataforma_educativa> db.materias.insertMany([
|   {
|     nombre: "Diseño de Sistemas",
|     recursos_ids: [101, 102]
|   },
|   {
|     nombre: "Arquitectura de Software",
|     recursos_ids: [102, 103]
|   }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69cf4324741aa807bf3682dc'),
    '1': ObjectId('69cf4324741aa807bf3682dd')
  }
}
```

Consultar en cuántas materias se usa un recurso específico (ej. el 102)

```
Atlas atlas-u9mhvu-shard-0 [primary] plataforma_educativa> const conteo = db.materias.countDocuments({ recursos_ids: 102
 });

Atlas atlas-u9mhvu-shard-0 [primary] plataforma_educativa> print("El recurso 102 se usa en " + conteo + " materias.");
El recurso 102 se usa en 2 materias.
```

Listar los nombres de todas las materias que usan el recurso 101

```
Atlas atlas-u9mhvu-shard-0 [primary] plataforma_educativa> db.materias.find(
|   { recursos_ids: 101 },
|   { nombre: 1, _id: 0 }
| )
[ { nombre: 'Diseño de Sistemas' } ]
```

Agregar un nuevo recurso (103) a una materia existente ("Diseño de Sistemas")

```
Atlas atlas-u9mhvu-shard-0 [primary] plataforma_educativa> db.materias.updateOne(
|   { nombre: "Diseño de Sistemas" },
|   { $addToSet: { recursos_ids: 103 } } // $addToSet evita duplicados si ya existía
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Materia actualizada

```
Atlas atlas-u9mhvu-shard-0 [primary] plataforma_educativa> db.materias.findOne({ nombre: "Diseño de Sistemas" })
{
  _id: ObjectId('69cf4324741aa807bf3682dc'),
  nombre: 'Diseño de Sistemas',
  recursos_ids: [ 101, 102, 103 ]
}
```

Se concluye con la normalización relativa, para favorecer la reutilización de activos digitales.

Código en FORMA COPY PASTE

```
// a) Crear la colección 'recursos' con datos reutilizables
db.recursos.insertMany([
  { _id: 101, titulo: "Manual de MongoDB", tipo: "PDF", link: "utn.edu/mongo.pdf" },
  { _id: 102, titulo: "Video: Modelado NoSQL", tipo: "Video", link: "youtube.com/nosql" },
  { _id: 103, titulo: "Whitepaper Cloud Computing", tipo: "PDF", link: "aws.com/cloud.pdf" }
])

// b) Crear la colección 'materias' referenciando los IDs de los recursos
db.materias.insertMany([
  {
    nombre: "Diseño de Sistemas",
    recursos_ids: [101, 102]
  },
  {
    nombre: "Arquitectura de Software",
    recursos_ids: [102, 103]
  }
])

// c) Consultar en cuántas materias se usa un recurso específico (ej. el 102)
// Esto devuelve el número exacto de documentos que contienen ese ID
const conteo = db.materias.countDocuments({ recursos_ids: 102 });
print("El recurso 102 se usa en " + conteo + " materias.");

// d) Listar los nombres de todas las materias que usan el recurso 101
db.materias.find(
  { recursos_ids: 101 },
  { nombre: 1, _id: 0 }
)

// e) Agregar un nuevo recurso (103) a una materia existente ("Diseño de Sistemas")
db.materias.updateOne(
  { nombre: "Diseño de Sistemas" },
  { $addToSet: { recursos_ids: 103 } } // $addToSet evita duplicados si ya existía
)

// Verificación final: Ver cómo quedó la materia actualizada
db.materias.findOne({ nombre: "Diseño de Sistemas" })
```